

Guía General del Proyecto SCAFAME y Acceso a la Base de Datos PostgreSQL

Índice

1. Guía General del Proyecto SCAFAME	3
1.1. Descripción General	3
1.2. Estructura Principal del Repositorio	3
1.3. Stack Tecnológico	3
1.3.1. Frontend	3
1.3.2. Backend	3
1.3.3. Infraestructura	4
1.4. Módulos Funcionales	4
1.4.1. Usuarios	4
1.4.2. Inventario	4
1.4.3. Entradas y Retiros	4
1.4.4. Historial y Reportes	4
1.4.5. Dashboard	4
1.5. Rutas Principales del Frontend	4
1.6. Endpoints Principales del Backend	5
1.7. Modelo de Datos (Entidades)	5
1.8. Imágenes de Productos	6
1.9. Ejecución en Desarrollo	6
1.9.1. Backend	6
1.9.2. Frontend	6
1.10. Despliegue con Docker	6
1.11. Variables y Configuración Clave	6
1.11.1. Backend	6
1.11.2. Frontend	7
1.12. Documentos Recomendados del Repositorio	7
1.13. Buenas Prácticas Operativas	7
1.14. Estado y Evolución	7
2. Guía de Acceso a la Base de Datos PostgreSQL por SSH y DBeaver	7
2.1. Objetivo	7
2.2. Estado Actual Validado	7
2.3. Requisitos	8
2.4. Conexión SSH a la VM	8
2.5. Verificar Contenedor y Puertos de PostgreSQL	8
2.6. Verificar Credenciales Reales de PostgreSQL	8
2.7. Configurar el Puerto 5432 en el Archivo YML	8
2.8. Conectar DBeaver desde la VM	9
2.9. Habilitar SSH en la VM (para túnel desde Windows)	9
2.10. Conectar desde Windows a la BD de la VM (túnel SSH)	9
2.11. Conectar DBeaver en Windows usando el túnel	10
2.12. Troubleshooting Rápido	10

2.12.1. Error: Connection refused en DBeaver de la VM	10
2.12.2. Error: Connection refused en túnel SSH desde Windows	10
2.12.3. Error: Password incorrecta SSH	10
2.12.4. DBeaver conecta en VM pero no en Windows	10
2.13. Seguridad Recomendada	10
2.14. Comandos a Evitar para no Perder Datos	10
2.15. Backup Previo Recomendado	10
2.16. Checklist Operativo Corto	11

1. Guía General del Proyecto SCAFAME

1.1. Descripción General

SCAFAME es un sistema de gestión de inventario y procesos internos, compuesto por:

- Frontend web en Angular.
- Backend API en NestJS.
- Base de datos PostgreSQL.
- Despliegue en contenedores Docker (backend, frontend, db y nginx).

El sistema cubre gestión de productos, categorías, unidades, entradas/salidas de inventario, reportes, historial y administración de usuarios.

1.2. Estructura Principal del Repositorio

- `CODE/scafame.frontend`: Aplicación Angular.
- `CODE/scafame.backend`: API NestJS + TypeORM.
- `CODE/docker-compose.yml`: Orquestación principal de servicios en Docker.
- `CODE/deploy frontend`: Imagen y configuración de Apache para servir frontend compilado.
- `CODE/nginx`: Configuración de reverse proxy.
- `DOCUMENTATION`: Documentación funcional y técnica.
- `QR GENERATION APP`: Utilitario independiente para generación de QR.

1.3. Stack Tecnológico

1.3.1. Frontend

- Angular 20
- RxJS
- TailwindCSS
- ngx-toastr
- Chart.js
- ExcelJS

1.3.2. Backend

- NestJS 11
- TypeORM 0.3
- PostgreSQL (driver pg)
- JWT/Passport para autenticación
- Multer para subida de imágenes
- ExcelJS para exportes

1.3.3. Infraestructura

- Docker Compose
- PostgreSQL 16
- Nginx + Apache (frontend servido como contenido estático)

1.4. Módulos Funcionales

1.4.1. Usuarios

- Alta, edición y eliminación de usuarios.
- Roles: `admin`, `manager`, `user`.
- Cambio de contraseña.

1.4.2. Inventario

- Visualización general de productos.
- Filtros por texto y categoría.
- Gestión de productos, categorías y unidades.
- Soporte de imagen por producto.

1.4.3. Entradas y Retiros

- Solicitud de productos para entradas.
- Solicitud y gestión de retiros.
- Revisión de solicitudes antes de registrar.

1.4.4. Historial y Reportes

- Historial de movimientos.
- Detalle de reportes.
- Exportación a Excel/PDF en vistas operativas.

1.4.5. Dashboard

- KPIs de inventario.
- Gráfico de stock por categoría/producto.
- Exportación de reporte filtrado.

1.5. Rutas Principales del Frontend

Rutas base:

- `/auth/login`
- `/home`
- `/users`
- `/inventory`

- /dashboard
- /removals
- /entries
- /history
- /modify

Rutas de usuarios:

- /users/list
- /users/add
- /users/edit
- /users/delete

1.6. Endpoints Principales del Backend

Controladores:

- auth
- users
- products
- product-categories
- units
- reports

Ejemplos de uso frecuentes:

- GET /products
- POST /products
- PATCH /products/:id
- GET /reports
- POST /reports
- GET /users
- POST /users

1.7. Modelo de Datos (Entidades)

Entidades principales configuradas en TypeORM:

- User
- Product
- Unit
- ProductCategory
- Report

- `ProductReport`

Relaciones clave:

- `Product` pertenece a `ProductCategory` y opcionalmente a `Unit`.
- `Report` se relaciona con productos a través de `ProductReport`.

1.8. Imágenes de Productos

- Las imágenes se sirven desde `/uploads`.
- En backend, la carga de archivos se maneja con `Multer`.
- Para despliegue en contenedor, la carpeta de uploads debe persistirse con volumen.

1.9. Ejecución en Desarrollo

1.9.1. Backend

Desde `CODE/scafame_backend`:

```
npm install
npm run start:dev
```

1.9.2. Frontend

Desde `CODE/scafame_frontend`:

```
npm install
npm start
```

1.10. Despliegue con Docker

Desde `CODE`:

```
docker compose build
docker compose up -d
```

Servicios esperados:

- `db` (PostgreSQL)
- `backend` (NestJS)
- `frontend` (Apache con build Angular)
- `nginx` (proxy)

1.11. Variables y Configuración Clave

1.11.1. Backend

- `DB_HOST`
- `DB_PORT`
- `DB_USER`
- `DB_PASSWORD`
- `DB_NAME`
- `NODE_ENV`

1.11.2. Frontend

- `API_BASE` en `src/environments/environment.prod.ts`.
- Resolución de imágenes via `image-url.pipe.ts`.

1.12. Documentos Recomendados del Repositorio

- `DOCUMENTATION/Software_Requirements_Specification_SCAFAME_Optimization_Addendum.md`
- `DOCUMENTATION/Guia_Acceso_Base_Datos_VM.md`

1.13. Buenas Prácticas Operativas

- Validar `npm run build` antes de despliegue frontend.
- No continuar despliegue remoto si el build falla.
- Evitar `docker compose down -v` en entornos con datos que se deben conservar.
- Respalidar BD antes de cambios de infraestructura.

1.14. Estado y Evolución

El proyecto ha tenido una fase activa de optimizaciones en:

- Filtros y búsquedas.
- Paginación/infinite scroll.
- Exportes de reporte.
- Estabilidad de imágenes.
- Mejoras de UI/UX en tablas y dashboard.

2. Guía de Acceso a la Base de Datos PostgreSQL por SSH y DBeaver

2.1. Objetivo

Documentar el proceso para:

- Exponer PostgreSQL solo hacia localhost de la VM.
- Conectarse desde Windows a la BD de la VM de forma segura mediante túnel SSH.
- Visualizar la base de datos con DBeaver en la VM o desde Windows.

2.2. Estado Actual Validado

- Contenedor de BD: `Scafame-db`.
- Motor: PostgreSQL.
- Base: `scafame_db`.
- Usuario: `admin`.
- Clave: `admin1234`.
- DBeaver en VM: conexión funcional.

2.3. Requisitos

- VM Linux con Docker y Docker Compose.
- Proyecto desplegado en la VM.
- DBeaver instalado en la VM.
- (Opcional) OpenSSH Server instalado en la VM para túnel desde Windows.

2.4. Conexión SSH a la VM

Conexión desde Windows:

```
ssh administrador@192.168.11.5
```

2.5. Verificar Contenedor y Puertos de PostgreSQL

Ejecutar:

```
docker ps --format "table - {{.Names}}\t {{.Ports}}"
```

Interpretación:

- `Scafame-db 5432/tcp` significa que PostgreSQL está expuesto solo dentro del entorno Docker.
- Si se desea acceso desde la VM o por túnel SSH, se recomienda publicar el puerto solo a loopback.

2.6. Verificar Credenciales Reales de PostgreSQL

Ejecutar:

```
docker exec Scafame-db env | grep -E "POSTGRES_DB|POSTGRES_USER|POSTGRES_PASSWORD"
```

Salida esperada (ejemplo actual):

```
POSTGRES_DB=scafame_db
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin1234
```

2.7. Configurar el Puerto 5432 en el Archivo YML

En el servicio de base de datos del `docker-compose.yml`, usar este mapeo:

```
ports:
- "127.0.0.1:5432:5432"
```

Esto permite:

- Conectarse desde la propia VM a `localhost:5432`.
- Crear un túnel SSH desde Windows sin exponer PostgreSQL a toda la red.

Aplicar cambios:

```
docker compose up -d --force-recreate db
docker ps --format "table - {{.Names}}\t {{.Ports}}" | grep -i db
```

Resultado esperado:

```
Scafame-db    127.0.0.1:5432->5432/tcp
```


2.8. Conectar DBeaver desde la VM

Configurar conexión PostgreSQL con:

- Host: `localhost`
- Port: `5432`
- Database: `scafame_db`
- User: `admin`
- Password: `admin1234`
- SSL: `disable` (si el driver lo solicita)

Prueba rápida de consulta:

```
SELECT * FROM product LIMIT 20;
```

Para visualizar relaciones en DBeaver (ERD):

- Abrir la conexión PostgreSQL.
- Expandir Schemas -¿public -¿Tables.
- Seleccionar varias tablas relevantes o el esquema completo.
- Click derecho -¿ER Diagram.

2.9. Habilitar SSH en la VM (para túnel desde Windows)

Instalar y activar:

```
sudo apt update
sudo apt install -y openssh-server
sudo systemctl enable --now ssh.socket
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

Notas:

- En Ubuntu puede aparecer `ssh.service inactive (dead)` con `ssh.socket` activo.
- Si el log indica `Server listening on ... port 22`, SSH está operativo.

2.10. Conectar desde Windows a la BD de la VM (túnel SSH)

En PowerShell de Windows:

```
ssh -N -L 5432:127.0.0.1:5432 administrador@192.168.11.5
```

- Aceptar huella de host en la primera conexión (`yes`).
- Mantener la terminal abierta mientras se use la conexión.

Si el puerto 5432 está ocupado en Windows:

```
ssh -N -L 15432:127.0.0.1:5432 administrador@192.168.11.5
```

Y conectar desde Windows a `localhost:15432`.

2.11. Conectar DBeaver en Windows usando el túnel

- Host: localhost
- Port: 5432 (o 15432 si se usó ese)
- Database: `scafame_db`
- User: `admin`
- Password: `admin1234`

2.12. Troubleshooting Rápido

2.12.1. Error: Connection refused en DBeaver de la VM

Causa común: DB no mapeada al host de la VM. Acción: revisar `ports` en `db` y recrear contenedor.

2.12.2. Error: Connection refused en túnel SSH desde Windows

Causa común: SSH no activo en VM. Acción:

```
sudo systemctl start ssh
sudo systemctl start ssh.socket
ss -tulpen | grep :22
```

2.12.3. Error: Password incorrecta SSH

- Verificar usuario real con `whoami`.
- Probar login local: `ssh administrador@localhost`.
- Si falla, resetear clave: `sudo passwd administrador`.

2.12.4. DBeaver conecta en VM pero no en Windows

- Revisar que el túnel SSH siga abierto.
- Validar puerto local correcto (5432 o 15432).

2.13. Seguridad Recomendada

- No publicar 5432:5432 en 0.0.0.0 salvo necesidad puntual.
- Preferir 127.0.0.1:5432:5432 + túnel SSH.
- Usar contraseñas fuertes y rotación periódica.

2.14. Comandos a Evitar para no Perder Datos

No ejecutar:

```
docker compose down -v
docker volume rm <volumen>
```

2.15. Backup Previo Recomendado

Antes de cambios de infraestructura:

```
mkdir -p ~/backups
docker exec Scafame-db pg_dump -U admin -d scafame_db > ~/backups/scafame_db_$(date +%Y%m%d_%H%M%S).sql
ls -lh ~/backups
```

2.16. Checklist Operativo Corto

1. DB arriba y saludable (`docker ps`).
2. Credenciales confirmadas (`docker exec ... env`).
3. DBeaver en VM conectando a localhost:5432.
4. SSH activo en VM para túnel.
5. Túnel abierto en Windows.
6. DBeaver en Windows conectando por localhost.